

■ 2.2.15 Advanced Topic: Associating Rules with Different Objects

When you make a definition like $f[x_] = x^2$, it is pretty clear that this should be associated with the “object” f , and, for example, displayed when you type `?f`.

But what should a definition like $g[x_] + g[y_] := g[x y]$ be associated with? The full form of the left-hand side is `Plus[g[x_], g[y_]]`. Looking at this form, one might think of associating the definition with `Plus`. The way *Mathematica* is set up, this would certainly work. However, it is not the best thing to do, either from the point of organization, or efficiency.

When you see a mathematical formula like $g(x) + g(y) = g(xy)$, you usually think of it as a relation concerning the object g , not a relation about addition. In this case, it would be more natural to associate a *Mathematica* definition such as $g[x_] + g[y_] := g[x y]$ with g than with `Plus`.

$g/: lhs := rhs$ define a transformation rule to be associated with the “object” g

Defining a rule to be associated with a particular object.

This rule is associated with f . `In[1]:= f[g[x_], g[y_]] := rhs[x, y]`

This shows the rules associated with f . `In[2]:= ?f`

$$\frac{f}{f/: f[g[x_], g[y_]] := rhs[x, y]}$$

The $g/:$ tells *Mathematica* to associate this rule with g , rather than with `Plus`. `In[2]:= g/: g[x_] + g[y_] := rhs[x, y]`

Here are the rules associated with g . `In[3]:= ?g`

$$\frac{g}{g/: g[x_] + g[y_] := rhs[x, y]}$$

The rule you gave is used. `In[3]:= g[a] + g[b]`
`Out[3]= rhs[a, b]`

You can also associate this rule with g . `In[4]:= g/: f[g[x_], g[y_]] := rhs[x, y]`

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

Now both rules appear.

```
In[5] := ?g
g
g/: g[x_] + g[y_] := rhs[x, y]
g/: f[g[x_], g[y_]] := rhs[x, y]
```

The ability to associate rules with different objects is very important when you use *Mathematica* expressions to represent “packets of data”.

You might want to introduce a mathematical object called `quat` that has special addition and multiplication properties. One way to do this would be to set up special addition and multiplication functions for `quat` objects. A much more convenient way to do it, however, is to use the standard addition and multiplication functions, but to add special rules associated with `quat` objects.

Some rules you could add for `quat` objects would be `quat/: quat[x_] + quat[y_] := value` and `quat/: quat[x_] quat[y_] := value`. You can think of these rules as “extending the domain” of addition and multiplication to include `quat` objects.

(If you are familiar with other computer languages, you may think of `quat` as specifying a “type”. Assignments associated with `quat` can then be used to “extend” or “overload” operators to allow for arguments of type `quat`. From the point of view of object-oriented systems, you could think of `quat` as corresponding to a type of “object”. The definitions you give for `quat` then specify “methods” associated with different operations or “messages”.)

Another case where you should use `/:` is in building up “databases” that give a variety of properties for a particular object. A sequence of definitions like `area[pentagon] = value`, `perimeter[pentagon] = value`, ..., could be associated with each of the quantities `area`, `perimeter`, and so on. It is much more natural, however, to use `/:` to associate all these definitions with the object `pentagon`.

This associates the definition with the object `square`.

```
In[5] := square/: area[square] = 1
Out[5] = 1
```

Here is another definition, again associated with `square`.

```
In[6] := square/: perimeter[square] = 4
Out[6] = 4
```

Here are the definitions for `square` that you have given.

```
In[7] := ?square
square
square/: area[square] = 1
square/: perimeter[square] = 4
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

You can use `/:` to associate a particular assignment with any symbol that appears in the left-hand side. A very common case is one where the assignment should be associated with the argument g in an expression like $f[g]$ or $f[g[\text{arguments}]]$. You can save typing $g/:$ by using the special assignment operators `^=` and `^:=`.

$f[g]^{\wedge}=\text{value}$ or $f[g[\text{args}]]^{\wedge}=\text{value}$	make assignments to be associated with g , rather than f
$f[g]^{\wedge}:=\text{value}$ or $f[g[\text{args}]]^{\wedge}:=\text{value}$	make delayed assignments associated with g
$f[\text{arg}_1, \text{arg}_2, \dots]^{\wedge}=\text{value}$	make assignments associated with the heads of <i>all</i> the arg_i

Shorter ways to make assignments associated with function arguments.

Using `^=` makes *Mathematica* associate the assignment with the argument `triangle`, rather than the function `area`.

`In[7]:= area[triangle] ^= 1/2`

`Out[7]=  $\frac{1}{2}$`

If you have a function with several arguments, `^=` makes assignments associated with *each* of the arguments. This is particularly useful when all the arguments have the same head, as in the case of `quat[x_] + quat[y_] ^= value`. You should be careful not to add extra definitions by mistake when different arguments have different heads.